Lab 9: Reading the IPv4 Header with Wireshark

Computer Networks · Week 9 · Network Layer / IPv4

What this lab does

In this lab, you read IPv4 headers in Wireshark, from a capture provided to you, and you compute in code the two things Wireshark shows you the result of but not the working: the header checksum and fragment reassembly. The capture contains a clean packet, a packet that was corrupted in transit, and one datagram that was split into three fragments.

Wireshark already decodes every header field for you, so you do not write a parser. You write only two short functions, for the two quantities Wireshark hides the arithmetic of: verifying the header checksum, and working out how a fragmented datagram reassembles. Everything else you read directly in Wireshark, and the lab's payoff is the reconciliation: your own numbers agree with the analyser's.

You will:

- open a capture in Wireshark and read the IPv4 header fields of a packet;
- verify an IPv4 header checksum in code, and show that the corrupted packet fails it;
- compute how a fragmented datagram reassembles from its fragment offset and flags;
- confirm both computations against what Wireshark reports.

Setup

Install Wireshark and open the provided file `lab8_ipv4.pcap`. For the two functions, use any C++ compiler or Python 3.

One Wireshark setting matters. By default Wireshark often does not validate the IPv4 header checksum (because real network cards compute it in hardware). Turn it on: Edit, Preferences, Protocols, IPv4, and enable "Validate the IPv4 header checksum." Without this, the corrupted packet in Part B will not be flagged.

What the capture contains

Five frames:

1. a clean UDP datagram with a valid header checksum;
2. the same datagram after its TTL was altered in transit, with the checksum left unchanged, so the checksum is now stale; 3, 4, 5. one datagram that was fragmented into three fragments, all sharing the same Identification.

Conventions and published values

For the two code tasks you are given the raw header bytes directly, so the functions do not depend on any capture tooling. All values are hexadecimal.

- IPv4 header layout (byte offsets): byte 0 holds Version and IHL; byte 1 holds DSCP and ECN; bytes 2 to 3 Total Length; bytes 4 to 5 Identification; bytes 6 to 7 the three Flags bits and the 13-bit Fragment Offset; byte 8 TTL; byte 9 Protocol; bytes 10 to 11 Header Checksum; bytes 12 to 15 Source Address; bytes 16 to 19 Destination Address.
- The header checksum is the 16-bit one's-complement sum over the header.
- The fragment offset is measured in units of 8 bytes.

Clean header (packet 1): `45 00 00 28 1C 46 00 00 40 11 DB 10 C0 A8 01 0A C0 A8 01 14`

Corrupted header (packet 2), TTL changed from 0x40 to 0x3F, checksum left as it was: `45 00 00 28 1C 46 00 00 3F 11 DB 10 C0 A8 01 0A C0 A8 01 14`

Fragment headers (packets 3, 4, 5):

Fragment	Identification	More-fragments flag	Fragment offset (units)	Total length	Payload
1	0x4A3F	1	0	1500	1480
2	0x4A3F	1	185	1500	1480
3	0x4A3F	0	370	60	40

Part A: Read the header in Wireshark

What you do: open the capture, select the clean packet, and record every IPv4 header field.

Your task:

1. Open `lab8_ipv4.pcap` and select packet 1. Expand the "Internet Protocol Version 4" section.
2. Record: version, header length (both the IHL value and the resulting byte count), total length, identification, the flags, fragment offset, TTL, protocol, header checksum, source address, destination address.
3. Confirm your readings against the clean header bytes given above. The expected values are version 4, IHL 5 (20 bytes), total length 40, identification 0x1C46, flags all clear, fragment offset 0, TTL 64, protocol 17 (UDP), checksum 0xDB10, source 192.168.1.10, destination 192.168.1.20.

Part B: Verify the checksum (code, then Wireshark)

What you do: implement the header-checksum check, run it on the clean and corrupted headers, then confirm the result in Wireshark.

Background: the IPv4 header checksum is formed by summing the header as 16-bit words in one's-complement arithmetic, where any overflow out of 16 bits is carried back in, and then taking the complement of the result. A receiver recomputes the same sum over the whole header, including the checksum field, and a valid header folds to all ones (0xFFFF). The checksum covers the header only, not the payload.

Your task:

1. Implement `verify_ip_checksum(header)` that returns the stored checksum, the checksum you compute from the other fields, and whether the header is valid.
2. Run it on the clean header. It must report stored 0xDB10, computed 0xDB10, valid.
3. Run it on the corrupted header. It must report stored 0xDB10, computed 0xDC10, invalid. The single changed byte (TTL) makes the stored checksum wrong.
4. In Wireshark, with checksum validation enabled, confirm that packet 2 is marked with a header-checksum error and packet 1 is not.

Part C: Fragment reassembly (Wireshark, then code)

What you do: read the three fragments in Wireshark, then compute how they reassemble and confirm it.

Background: a datagram too large for a link is split into fragments that share one Identification. Each fragment carries a fragment offset, in units of 8 bytes, giving where its payload sits in the original datagram, and a more-fragments flag that is 1 on every fragment except the last. The receiver places each fragment's payload at its offset and knows the datagram is complete when the pieces are contiguous from offset 0 and the final piece has the more-fragments flag clear.

Your task:

1. In Wireshark, select each of the three fragments and read its Identification, more-fragments flag, and fragment offset.
2. Implement `reassemble(fragments)` that takes the fragments and returns, for each, the byte range it covers in the original datagram, the total reassembled length, and whether the datagram is complete.
3. Confirm the ranges are 0 to 1479, 1480 to 2959, and 2960 to 2999, the total length is 3000 bytes, and the datagram is complete.
4. Confirm against Wireshark, which reassembles the fragments itself and shows the full datagram on the last fragment.

Part D: Analysis

What you do: explain the behaviour behind the two computations.

Your task:

1. Name the header field that changes at every router (TTL), and explain why the header checksum must therefore be recomputed at every hop. Relate this to the corrupted packet in Part B.
2. Using the fragment offsets, explain why every fragment except the last must carry a payload whose length is a multiple of 8 bytes.
3. Explain why the IPv4 checksum covers only the header and not the payload, and what protects the payload instead.

What to submit

- Your source file (C++ or Python) with the two functions.
- Program output showing the checksum results on both headers and the reassembly result on the three fragments, matching the values above.
- Your Wireshark field readings for packet 1, and confirmation that packet 2 shows a checksum error.
- The written answers to Part D.

Appendix: the interface each program must implement

- `verify_ip_checksum(header) -> (stored, computed, valid)`, where `header` is the 20 header bytes.
- `reassemble(fragments) -> (ranges, total_length, complete)`, where each fragment is its fragment offset (in 8-byte units), its more-fragments flag, and its payload length in bytes.

The header checksum is a 16-bit one's-complement sum, folding carries back in; the fragment offset is in units of 8 bytes, so a payload byte position is the offset times 8.